

---

# ***Informe de Ambientes***

**Proyecto: Autogestión-Sena**

## ÍNDICE

1. Introducción
2. Objetivos del documento
3. Estructura de Ambientes en GitHub
  - 3.1. ¿Qué es un ambiente en control de versiones?
  - 3.2. Ambientes utilizados en Autogestión SENA
4. Flujo de trabajo (Workflow) por ambientes
  - 4.1. Ambiente *dev*
  - 4.2. Ambiente *qa*
  - 4.3. Ambiente *staging*
  - 4.4. Ambiente *main*
5. Uso de ambientes por repositorio
  - 5.1. Frontend Web (React + TS + Vite)
  - 5.2. Backend (Django REST Framework)
  - 5.3. Aplicación móvil (.NET MAUI)
6. Problemas encontrados y soluciones implementadas
7. Ejemplos con capturas (Frontend)
8. Conclusiones

## 1. Introducción

Este informe describe el manejo de ambientes utilizado en los repositorios de GitHub del proyecto **Autogestión SENA**.

La implementación de ambientes permite organizar el flujo de desarrollo, pruebas y despliegue, asegurando que cada cambio pase por un proceso de validación antes de ser liberado a producción.

El documento explica cómo se configuraron los ambientes, cómo se trabajó en cada uno y qué procedimientos se siguieron para garantizar una integración controlada de las funcionalidades.

## **2. Objetivos del documento**

- Describir los ambientes utilizados durante el desarrollo.
- Explicar el flujo de trabajo que se aplicó en GitHub.
- Mostrar cómo se implementó este manejo en el frontend, backend y móvil.
- Presentar ejemplos reales con comandos y ramas creadas.
- Documentar problemas encontrados y cómo afectaron el proceso.
- Dejar una guía clara para futuros desarrolladores del proyecto.

### 3. Estructura de Ambientes en GitHub

#### 3.1. ¿Qué es un ambiente en control de versiones?

Un ambiente es un espacio lógico que representa un **estado del software** dentro del ciclo de vida.

Cada ambiente permite evaluar el proyecto en diferentes etapas:

- **Desarrollo**
- **Pruebas**
- **Preproducción**
- **Producción**

Esto evita mezclar código experimental con código estable y garantiza calidad en los despliegues.

#### 3.2. Ambientes utilizados en Autogestión SENA

El proyecto definió **cuatro ambientes principales**:

| Ambiente | Propósito                                                 |
|----------|-----------------------------------------------------------|
| dev      | Desarrollo diario, nuevas funcionalidades y correcciones. |
| qa       | Pruebas funcionales y validaciones internas.              |
| staging  | Simulación de producción antes del lanzamiento.           |
| main     | Versión estable y lista para despliegue final.            |

## 4. Flujo de trabajo (Workflow) por ambientes

A continuación se documenta el flujo exacto utilizado para cada ambiente, basado en ramas por HU (Historia de Usuario).

### 4.1. Ambiente *dev*

Es el primer ambiente de integración.

#### Procedimiento:

```
git pull origin dev
git switch dev
git switch -c HU-01-dev
# → desarrollo
# → commit y push
Crear MR desde: HU-01-dev → dev
```

Se cierra el desarrollo cuando la rama **dev** queda actualizada con la HU aprobada.

### 4.2. Ambiente *qa*

Para pruebas funcionales y validaciones.

#### Procedimiento:

```
git pull origin qa
git switch qa
git switch -c HU-01-qa
# → desarrollo QA
Crear MR: HU-01-qa → qa
```

Se cierra una vez la HU se aprueba en las pruebas.

### 4.3. Ambiente *staging*

Antes de llevar a main, staging permite probar el sistema completo.

**Procedimiento:**

```
git pull origin staging
git switch staging
git switch -c HU-01-staging
# → ajustes finales
Crear MR: HU-01-staging → staging
```

Esta rama simula el entorno real antes de producción.

#### **4.4. Ambiente main**

Es la rama más importante: código listo para publicación.

**Procedimiento:**

```
git pull origin main
git switch main
git switch -c release.1.1
```

Dentro de esa release ingresan todas las HU del sprint:

```
HU-01-release.1.1
HU-02-release.1.1
HU-03-release.1.1
...
```

Luego:

```
Crear MR: release.1.1 → main
```

Nunca se debe trabajar directamente sobre main.

## **5. Uso de ambientes por repositorio**

### **5.1. Frontend Web (React + TS + Vite)**

Este repositorio fue el que sí logró implementar exitosamente todo el manejo de ambientes.

- Se trabajaron ramas dev, qa, staging y main.
- Se crearon ramas por HU por cada ambiente.
- Se hicieron merges controlados.
- Mantiene commits limpios y un flujo ordenado.

### **5.2. Backend (Django REST Framework)**

En el backend se intentó implementar el mismo flujo, pero no se logró completamente.

Razones:

- Dependencias rotas entre ramas.
- Migraciones de base de datos generaban conflictos.
- Diferencias entre entornos locales impedían mover código fácilmente.
- 
- Se trabajó más directo sobre dev y main.

Resultado: el manejo de ambientes no fue tan sólido como en frontend.

### **5.3. Aplicación Móvil (.NET MAUI)**

También se intentó el manejo de ambientes, pero surgieron fallos.

- La UI cambiaba constantemente.
- Problemas de sincronización entre repositorios.
- Se avanzó más lento debido al enfoque visual.

El flujo se implementó parcialmente.



## 6. Problemas encontrados y soluciones implementadas

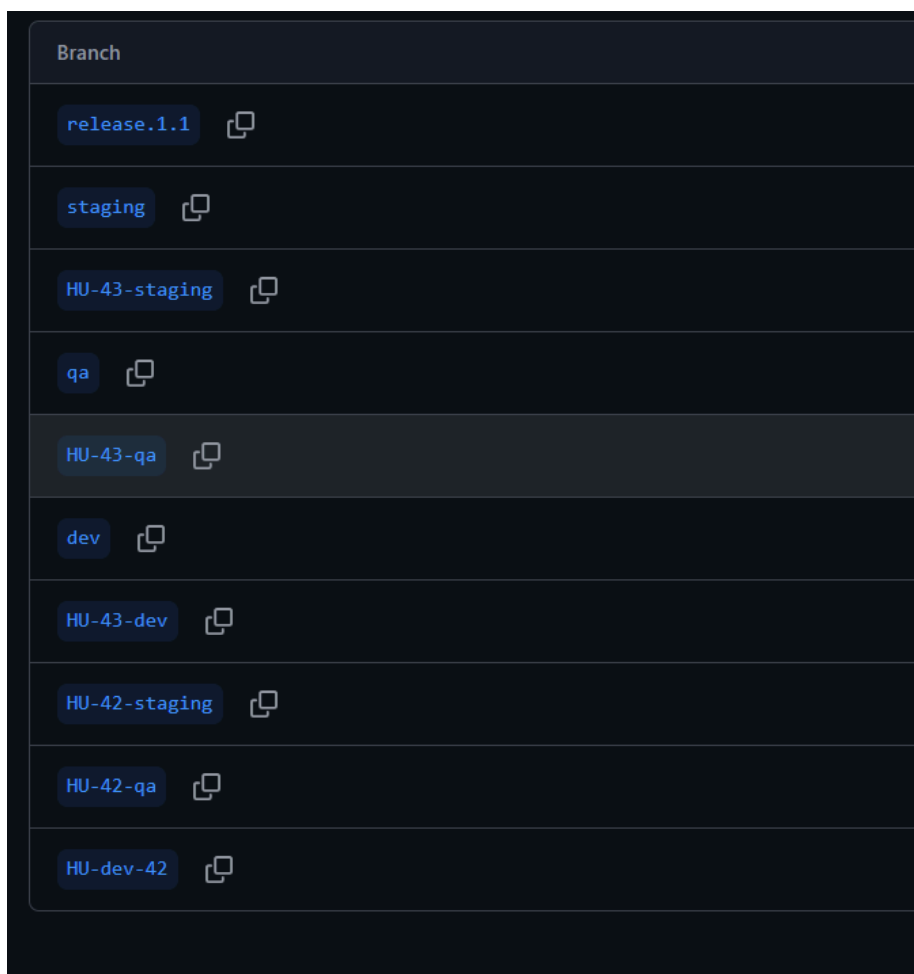
| Problema                                | Causa                             | Solución                                                |
|-----------------------------------------|-----------------------------------|---------------------------------------------------------|
| Conflictos entre ramas                  | Migraciones y cambios simultáneos | Establecer un orden de commits y revisar antes de merge |
| Fallo en control de versiones del móvil | Cambios rápidos                   | Acordar estándares de commits                           |
| CI fallaba por dependencias             | Versiones diferentes              | Unificar versiones en equipo                            |
| Muchas ramas abiertas                   | Falta de cierre                   | Políticas de merge estrictas                            |

## 7. Ejemplos con capturas (Frontend)

### Aclaracion:

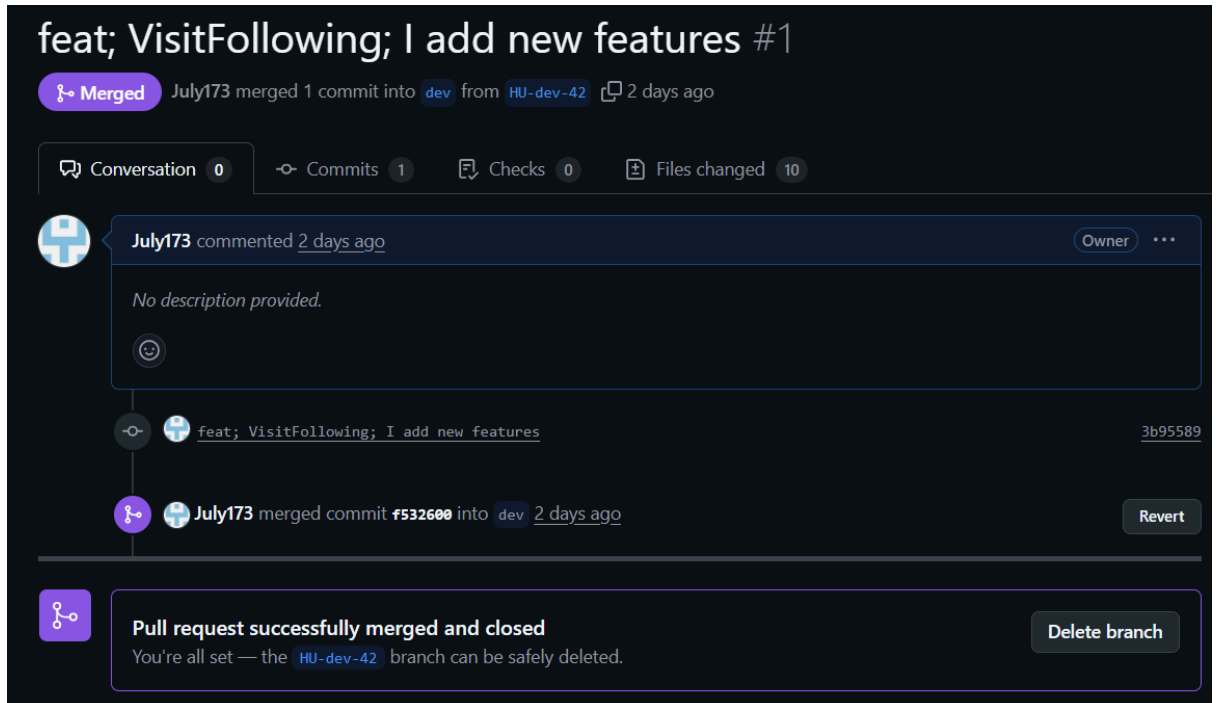
Durante el desarrollo se implementó el uso de cherry-pick para trasladar cambios específicos entre ramas y asegurar la correcta gestión de cada Historia de Usuario (HU), apoyándonos en la herramienta Fork para facilitar este proceso.

Debido a dificultades iniciales con el manejo de ambientes en los diferentes repositorios, se decidió consolidar un ambiente final en el frontend, aplicando correctamente el flujo dev → qa → staging → main. En este ambiente se integraron y documentaron los cambios finales y las HU aprobadas, con el objetivo de demostrar y evidenciar el proceso de gestión de ambientes que se aprendió e implementó durante el proyecto.

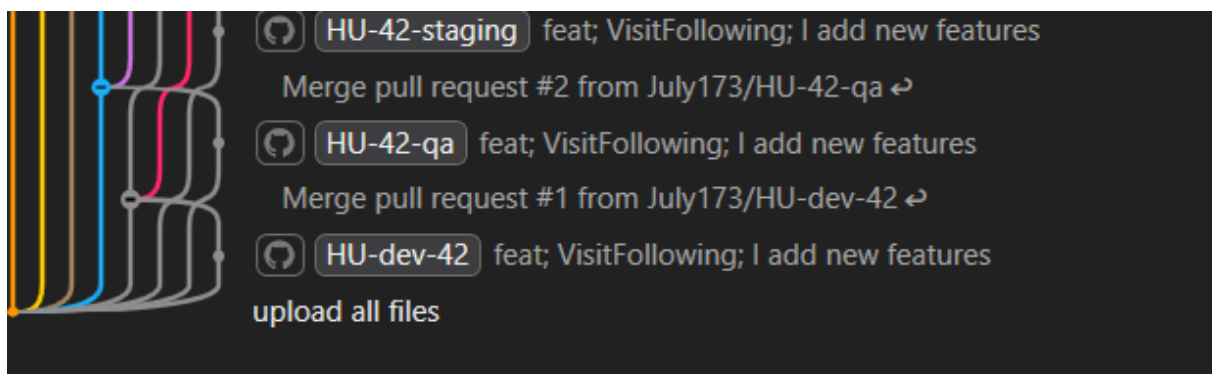


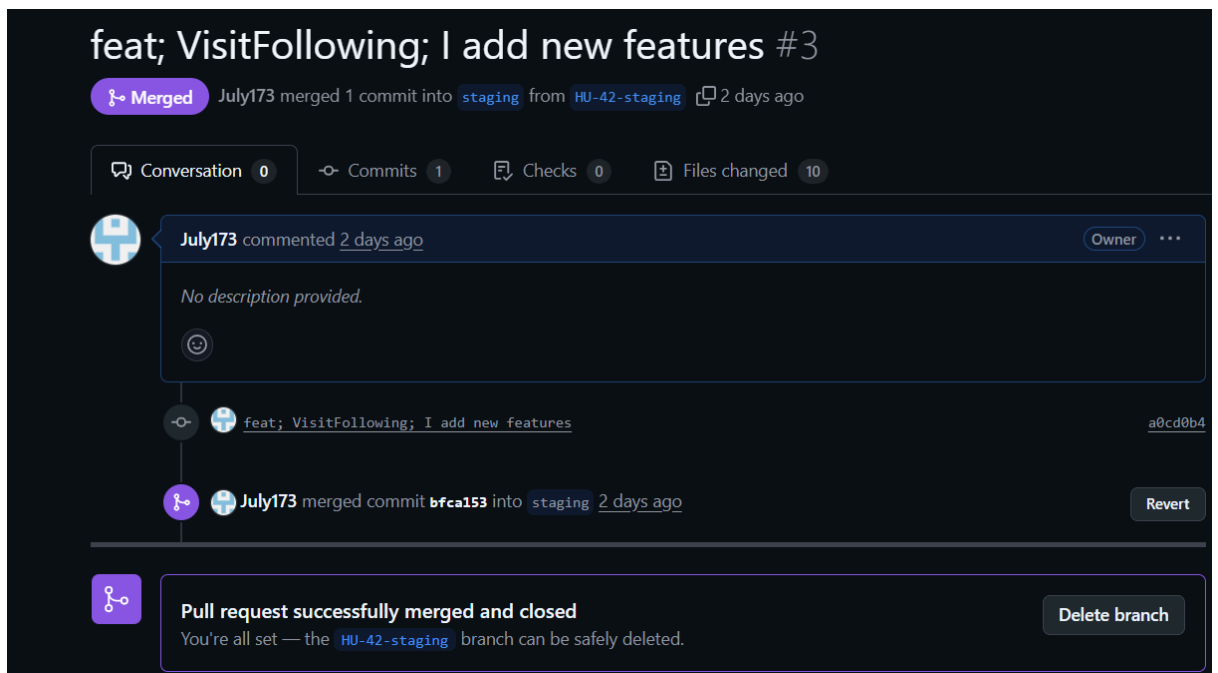
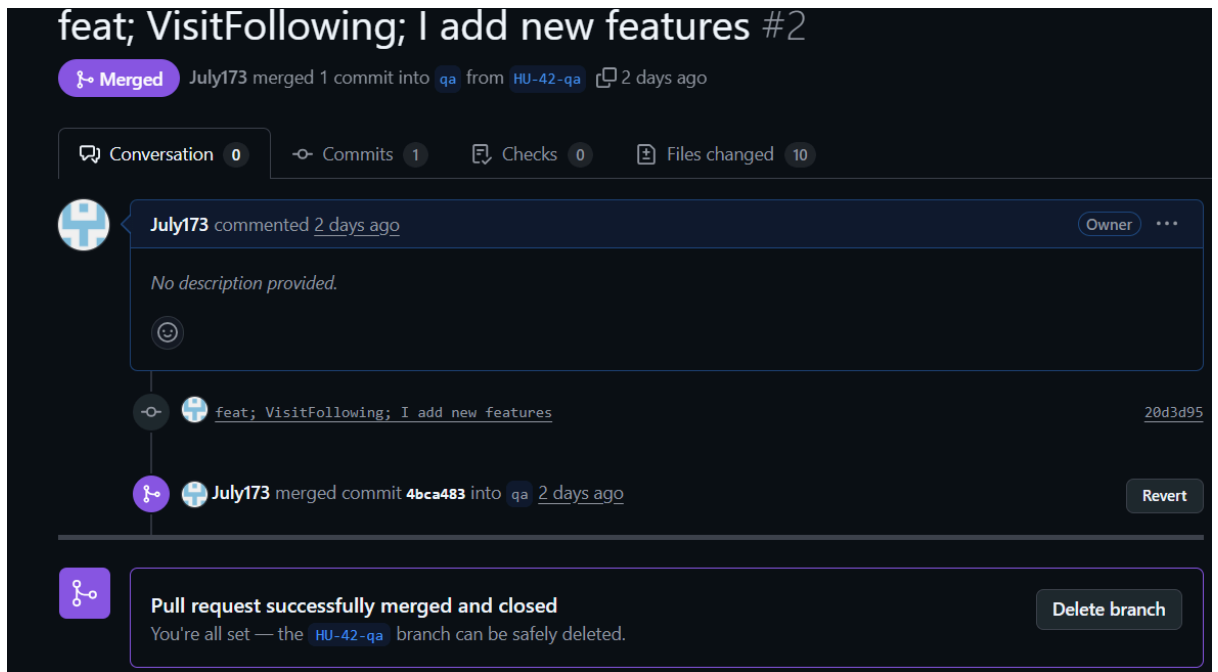
dev:

Se creo la HU-42-dev se hizo el proceso y se hizo el merge a la dev:



Posteriormente, los cambios desarrollados en la rama **HU-42-dev** fueron promovidos hacia la rama **HU-42-qa**, y luego hacia **HU-42-staging**, siguiendo el flujo de ambientes establecido. Durante este proceso se realizaron los **merge correspondientes entre cada HU y su rama principal**, es decir, primero desde la HU hacia **qa**, y luego desde la HU hacia **staging**, garantizando que cada nivel recibiera únicamente los cambios aprobados y validados para ese ambiente.

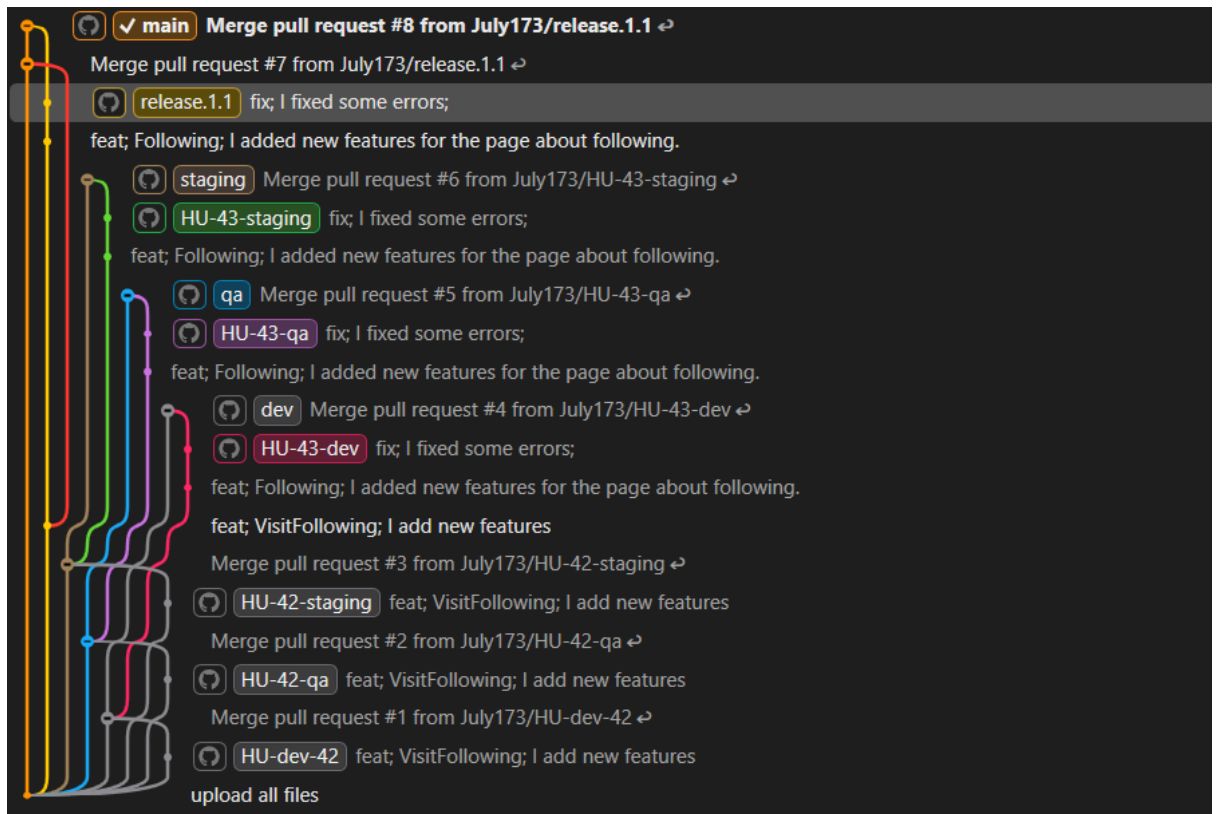




Posteriormente, se aplicó el mismo flujo de trabajo con la HU-43, trasladando los cambios desde la rama **HU-43-dev** hacia **HU-43-staging**, siguiendo las etapas correspondientes.

Una vez validadas las historias de usuario en los entornos previos, se procedió a generar los **release**, consolidando las funcionalidades aprobadas. Finalmente, estos

cambios fueron integrados en la rama **main**, completando el ciclo de publicación.



## Release.1.1 #8

**Merged** July173 merged 2 commits into `main` from `release.1.1` 1 hour ago

Conversation 0 Commits 2 Checks 0 Files changed 23

**July173** commented 1 hour ago Owner ...

No description provided.

**July173** added 2 commits yesterday

- feat; Following; I added new features for the page about following. [f11d3bc](#)
- fix; I fixed some errors; [f57ce65](#)

**July173** merged commit [631dedb](#) into `main` 1 hour ago Revert

## 8. Conclusiones

- El manejo de ambientes permitió un flujo organizado y controlado.
- El **frontend** fue el repositorio con implementación correcta y completa.
- El **backend y móvil** presentaron fallos, pero lograron mantener un flujo funcional.
- Este esquema de ambientes facilita revisiones, pruebas y despliegues seguros.
- Es una práctica recomendada para futuros desarrollos del Sistema Autogestión SENA.